Практическое занятие № 5

Цель работы

Пошагово выполнить первый цикл алгоритма шифрования ГОСТ 28147-89 в режиме простой замены, разобрать преобразование функции f(R, X) и на реальных двоичных данных увидеть, как из исходного блока формируется новый 32-битный результат R1.

Задание: Выполните первый цикл алгоритма шифрования ГОСТ 28147 89 в режиме простой замены. Для получения 64 бит исходного текста используйте восемь первых букв сообщения, составленного из своих фамилии и имени(если фамилия меньше восьми букв). Для получения ключа (256 бит) используйте любой текст, состоящий из 32 букв. Первый подключ должен содержать первые четыре буквы этого текста. Кодировка символов кириллицы приведена в таблице 3. Блок подстановки алгоритма шифрования ГОСТ 28147 89 приведен в таблице 4.

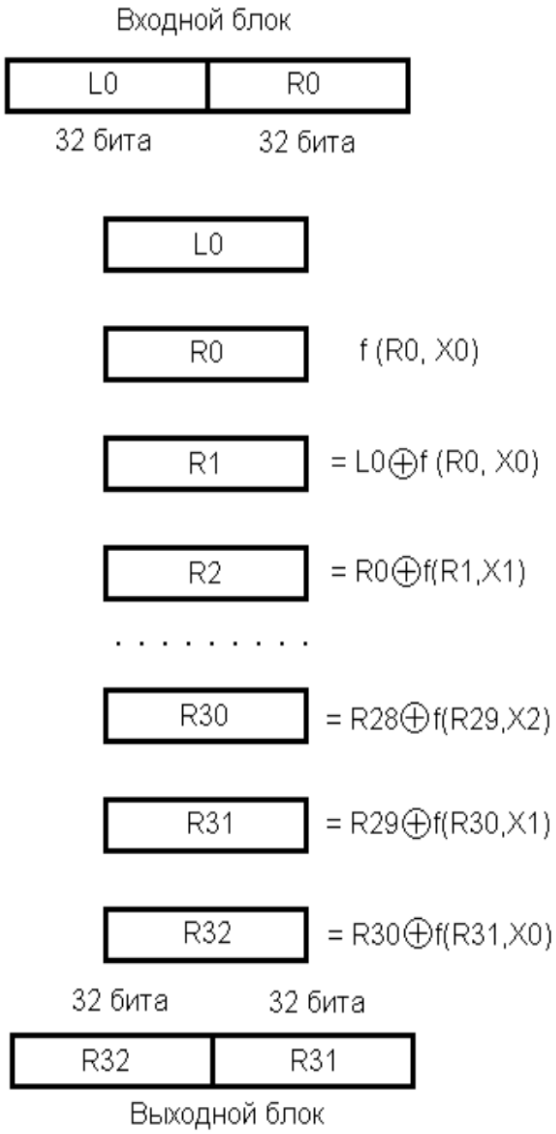
Рисунок 1 - Формулировка задания из методических указаний

Ход выполнения

Для расчета использован пример из методички: "ИНФОРМАЦ" как 64-битный исходный блок и "СИНТЕЗ передаточной функции цепи" как основа ключа. Первый подключ X0 содержит первые 4 буквы: "СИНТ".

Схема алгоритма перед расчетами

Схема ниже показывает основу цикла: блок делится на L0 и R0, правая часть проходит через функцию f(R, X), а затем по mod 2 объединяется с левой частью.



ıс. 3. Алгоритм шифрования ГОСТ 28147-89 (режим простой з
Знак ⊕ обозначает сложение по модулю 2.

Рисунок 2 - Общая схема шифрования ГОСТ 28147-89 в режиме простой замены

На этом месте алгоритм уже выглядит не как абстрактная формула, а как вполне понятная последовательность шагов: взяли R0, прогнали через f, потом объединили результат с L0.

Шаг 1. Перевод исходного текста и первого подключа в двоичный вид

Для начала нужно перевести символы кириллицы в двоичный вид по таблице кодировки из методички. Без этого дальше просто не из чего собирать L0, R0 и X0.

рис. 4. Функция преобразования *f(R,X)* в алгоритме ГОСТ 28147-89.

Таблица 3. Символы кириллицы (часть альтернативной кодовой таблицы ASCII)

№	Символ	Представление		№	Символ	Представление	
		Десятичное	Двоичное			Десятичное	Двоичное
1	А	192	11000000	18	С	209	11010001
2	Б	193	11000001	19	Т	210	11010010
3	В	194	11000010	20	У	211	11010011
4	Г	195	11000011	21	Ф	212	11010100
5	Д	196	11000100	22	Х	213	11010101
6	Е	197	11000101	23	Ц	214	11010110
7	Ж	198	11000110	24	Ч	215	11010111
8	З	199	11000111	25	Ш	216	11011000
9	И	200	11001000	26	Щ	217	11011001
10	Й	201	11001001	27	Ъ	218	11011010
11	К	202	11001010	28	Ы	219	11011011
12	Л	203	11001011	29	Ь	220	11011100
13	М	204	11001100	30	Э	221	11011101
14	Н	205	11001101	31	Ю	222	11011110
15	О	206	11001110	32	Я	223	11011111
16	П	207	11001111	33	пробел	32	00010000
17	Р	208	11010000				

Рисунок 3 - Таблица кодировки символов кириллицы, используемая в работе

Исходный текст

Символ	**Двоичный код**	**Hex**
И	11001000	C8
Н	11001101	CD
Ф	11010100	D4
О	11001110	CE
Р	11010000	D0
М	11001100	CC

А	11000000	C0
Ц	11010110	D6

Первый подключ X0

Символ	Двоичный код	Hex
С	11010001	D1
И	11001000	C8
Н	11001101	CD
Т	11010010	D2

Логи

```
Исходный текст:
И  11001000
Н  11001101
Ф  11010100
О  11001110
Р  11010000
М  11001100
А  11000000
Ц  11010110

Первый подключ X0:
С  11010001
И  11001000
Н  11001101
Т  11010010
```

После этого уже можно без гаданий собрать двоичные последовательности. На практике именно этот шаг часто съедает больше всего времени, потому что любая ошибка в одном бите потом уезжает во все следующие расчеты.

Шаг 2. Формирование L0, R0 и X0

Первые 64 бита исходного текста разбиваем на две части по 32 бита. Левая половина дает L0, правая половина дает R0. Первые 32 бита ключа дают X0.

Логи

```
L0: 11001000 11001101 11010100 11001110
R0: 11010000 11001100 11000000 11010110
X0: 11010001 11001000 11001101 11010010

L0 (hex): C8CDD4CE
R0 (hex): D0CCC0D6
X0 (hex): D1C8CDD2
```

Здесь уже видно, что алгоритм работает с двумя 32-битными половинами. То есть шифруется не весь блок целиком за один раз, а через классическую схему с левым и правым полублоком.

Шаг 3. Вычисление суммы R0 и X0 по mod 2^32

Теперь берем правую половину исходного блока и складываем ее с первым подключом по mod 2^32. Это первый этап внутри функции f(R, X).

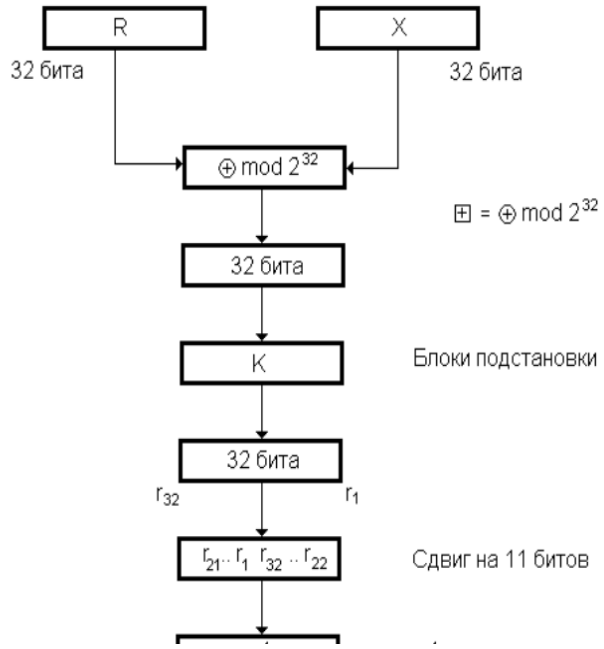


Рисунок 4 - Функция преобразования f(R, X)

Логи

```
R0: 1101 0000 1100 1100 1100 0000 1101 0110
X0: 1101 0001 1100 1000 1100 1101 1101 0010
Σ : 1010 0010 1001 0101 1000 1110 1010 1000
Σ (hex): A2958EA8
```

Результат суммы нужен не сам по себе. Это заготовка для следующего шага, где каждый 4-битный блок будет отдельно пропускаться через таблицу подстановки. По сути, здесь функция f только разгоняется.

Шаг 4. Преобразование в блоке подстановки

После сложения 32-битная последовательность разбивается на 8 тетрад по 4 бита. Дальше для каждой тетрады ищется номер строки в таблице подстановки. Номер столбца задается номером блока.

Таблица 4. Блок подстановки в алгоритме шифрования ГОСТ 28147-89

	8	7	6	5	4	3	2	1
0	1	13	4	6	7	5	14	4
1	15	11	11	12	13	8	11	10
2	13	4	10	7	10	1	4	9
3	0	1	0	1	1	13	12	2
4	5	3	7	5	0	10	6	13
5	7	15	2	15	8	3	13	8
6	10	5	1	13	9	4	15	0
7	4	9	13	8	15	2	10	14
8	9	0	3	4	14	14	2	6
9	2	10	6	10	4	15	3	11
10	3	14	8	9	6	12	8	1
11	14	7	5	14	12	7	1	12
12	6	6	9	0	11	6	0	7
13	11	8	12	3	2	0	7	15
14	8	2	15	11	5	9	5	5
15	12	12	14	2	3	11	9	3

Рисунок 5 - Таблица 4 и исходные данные примера из методички

Логи

```
Номера блоков:                  8    7    6    5    4    3    2    1
Блоки после сложения:        1010 0010 1001 0101 1000 1110 1010 1000
Номера строк в таблице:        10   2    9    5    8   14   10    8
Заполнение по таблице:          3   4    6   15   14    9    8    6
Результат подстановки:       0011 0100 0110 1111 1110 1001 1000 0110
```

Вот на этом шаге алгоритм наконец перестает быть просто арифметикой. Таблица подстановки вносит нелинейность, и именно за счет нее шифрование не сводится к тривиальным операциям сложения и XOR.

Шаг 5. Циклический сдвиг результата на 11 бит влево

Дальше результат подстановки циклически сдвигается на 11 бит влево. В методичке этот шаг приведен уже в готовом двоичном виде, его и берем дальше без изменений.

Логи

```
После сдвига на 11 бит влево:
1111 0100 1100 0011 0001 1010 0011 0111

f(R0, X0) = 1111 0100 1100 0011 0001 1010 0011 0111
```

Теперь функция f(R0, X0) полностью готова. То есть у нас уже есть тот самый 32-битный результат, который дальше нужно объединить с L0.

Шаг 6. Вычисление R1 = f(R0, X0) XOR L0

Последний шаг первого цикла: складываем результат функции f(R0, X0) с левой половиной L0 по mod 2. Здесь и появляется новый правый блок R1.

Таким образом, первые 64 бита определяют входную последовательность:

L0: 11001000 11001101 11010100 11001110
R0: 11010000 11001100 11000000 11010110

Следующие 32 бита определяют первый подключ:

X0: 11010001 11001000 11001101 11010010

I. Найдем значение функции преобразования f(R0,X0) (см. рис.4)

1). Вычисление суммы R0 и X0 по mod 2^{32}

R0: 1101 0000 1100 1100 1100 0000 1101 0110
X0: 1101 0001 1100 1000 1100 1101 1101 0010

1010 0010 1001 0101 1000 1110 1010 1000

2). Преобразование в блоке подстановки

Результат суммирования R0+X0 по mod 2^{32}:

1010 0010 1001 0101 1000 1110 1010 1000

преобразуем в блоке подстановки (см. таблицу 4). Для каждого четырехбитного блока вычислим его адрес в таблице подстановки. Номер блока соответствует номеру столбца, десятичное значение блока соответствует номеру строки в таблице. Таким образом, четвертый блок (1000) заменяется заполнением 8-ой строки и четвертого столбца в таблице подстановки (1110).

номера блоков

8	7	6	5	4	3	2	1
1010	0010	1001	0101	1000	1110	1010	1000

соответствующие номера строк в таблице подстановки

10	2	9	5	8	14	10	8

заполнение

3	4	6	15	14	9	8	6

результат

0011 0100 0110 1111 1110 1001 1000 0110

3) Циклический сдвиг результата п.2 на 11 бит влево

1111 0100 1100 0011 0001 1010 0011 0111

Таким образом, нашли значение функции f (R0,X0):

1111 0100 1100 0011 0001 1010 0011 0111

II. Вычисляем R1= f(R0,X0) ⊕L0.

Результат преобразования функции f(R0,X0) складываем с L0 по mod2:

L0: 1100 1000 1100 1101 1101 0100 1100 1110
f(R0,X0): 1111 0100 1100 0011 0001 1010 0011 0111

R1: 0011 1100 0000 1110 1100 1110 1111 1001

Рисунок 6 - Фрагмент методички с пошаговым вычислением первого цикла

Логи

```
L0:          1100 1000 1100 1101 1101 0100 1100 1110
f(R0,X0):    1111 0100 1100 0011 0001 1010 0011 0111
R1:          0011 1100 0000 1110 1100 1110 1111 1001

R1 (hex):    3C0ECEF9
```

Здесь уже получаем итог первого цикла. Если смотреть по-человечески, то исходный правый блок прошел через подключ, таблицу подстановки, циклический сдвиг и только после этого дал новый R1. Именно в этом и чувствуется "мясо" алгоритма.

Что шифровалось и что получилось после первого цикла

Позиция	Значение
Что шифровалось	64-битный блок из символов "ИНФОРМАЦ"
L0	11001000 11001101 11010100 11001110
R0	11010000 11001100 11000000 11010110
Первый подключ X0	11010001 11001000 11001101 11010010
Что получили	Новый правый блок R1 после первого цикла
R1	00111100 00001110 11001110 11111001
R1 в hex	3C0ECEF9
Если записать состояние после 1 цикла	L1 = R0, R1 = 3C0ECEF9

То есть в этой работе мы не доводили алгоритм до полного 32-циклового шифрования. Но уже после первого цикла хорошо видно, как из исходного блока начинает формироваться новый набор данных.

Что можно сравнить по режимам ГОСТ 28147-89

Раз в начале методичка перечисляет все режимы, логично коротко сравнить их между собой. Здесь разница не только в названии. Режим простой

замены хорош как база, но для больших массивов данных он слабее из-за независимого шифрования одинаковых блоков.

Режим	Что видно на практике	Короткий комментарий
Простая замена	Одинаковые блоки дают одинаковый результат	Подходит для короткой ключевой информации
Гаммирование	Вырабатывается гамма, не зависящая от данных	Удобнее для потока данных и маскировки повторов
Гаммирование с обратной связью	Гамма зависит от предыдущих шифрблоков	Лучше связывает соседние блоки между собой
Имитовставка	Проверяет целостность, а не заменяет шифрование	Нужна для контроля и обнаружения искажений

Для этой практики самый важный вывод такой: режим простой замены нужен как базовый кирпич. Остальные режимы строятся уже поверх него и закрывают те проблемы, которые возникают при работе с длинными данными.

Вывод

По ходу работы я последовательно разобрал весь первый цикл ГОСТ 28147-89. Сначала перевел символы "ИНФОРМАЦ" и "СИНТ" в двоичный вид, потом собрал из них L0, R0 и X0. Уже на этом шаге стало понятно, что ошибка даже в одном символе ломает всю дальнейшую цепочку вычислений.

Дальше получилось увидеть, как работает функция f(R, X) изнутри. Сложение R0 и X0 по mod 2^32 дает только промежуточную последовательность. Реальное усложнение начинается в блоке подстановки, где

каждая тетрада меняется по своей строке и своему столбцу. После этого циклический сдвиг на 11 бит окончательно перестраивает данные.

Главный результат практики: после первого цикла из исходных половин L0 и R0 был получен новый правый блок R1 = 00111100 00001110 11001110 11111001, или 3C0ECEF9 в шестнадцатеричном виде. Если записывать состояние после цикла целиком, то L1 просто становится равным старому R0, а новый R1 уже содержит результат всех преобразований.

Отдельно я для себя зафиксировал разницу между режимами алгоритма. Простая замена хорошо показывает внутреннюю механику ГОСТ, но для больших массивов данных одного такого режима мало. Гаммирование и гаммирование с обратной связью заметно практичнее, а режим имитовставки вообще отвечает уже за контроль целостности. Так что эта работа полезна именно тем, что на одном цикле дает понять основу, на которой потом строятся остальные режимы.